VDK/TLM Creation

Rapid Training

X-2025.06

Module 2: Lab C

Add Versioning Support to the SPI Controller

© 2025 Synopsys, Inc.

Table of Contents

Lab Overview 2

Task-1: Setup and Open the SPI project 2

Task-2: Add a Specification Version Control Parameter 3

Task-3: Updating the Model Version 3

What's Next 6

Lab Overview

In the previous lab Module 2 Lab B, we implemented the base functionality of the Serial Protocol Interface (SPI) controller peripheral – the task for this lab is to implement a newer version of the model with few additional features, and to add versioning information to the project to maintain multiple versions of the same model in a single TLMC project. This makes it easier to maintain and develop the model over time.

Please refer to the SPI Controller Specification for interface and behavior of this training example model.

Task-1: Setup and Open the SPI project

***Motivation**: In order to complete this training successfully there are few setup instructions that are required. This has to do with dependencies and configuration options required to complete the exercise.*

1. If you have completed the previous lab (Module 2 Lab B), you can continue with your existing TLM Creator project and skip reading the rest of this task and continue with `Task-2`.

   Otherwise, if want to start from a prepared starting point for this lab, use the following steps to create the starting point.

   i. Open a console and invoke Virtualizer studio from the Synopsys Virtual Prototyping installation.

   ```
   % source <installDir>/SLS/windows/setup.bat vi[rtualizer] (Windows)

   % source <installDir>/SLS/linux/setup.csh|sh vi[rtualizer] (Linux)
   ```

   This sourcing option picks up the Virtualizer *Elite* or *Base* license in that order, whichever is available. For more details on other options, refer to the VP documentation.

   i. Go to the `<Lab-data>/data/module_2` directory and start Virtualizer Studio.

   ```
   vs -d start_Module2LabC &
   ```

2. Close the welcome screen and import the starting point TLM Creator Project by clicking ***File*** → ***Import…*** menu option.

   i. In the ***Import*** dialog, select ***General*** → ***Existing Projects into Workspace*** to select what you want to import.

   ii. Click ***Next*** to get to the ***Import Projects*** dialog. Browse to the `SPICtrl` directory in the `start_Module2LabC` directory and click ***Finish*** (Make sure you are in the right directory – it's easy to end up in the wrong directory!).

   iii. Select the SPICtrl`.tlmc` file in the Project Explorer and click ***Generate Sources Code*** from the ***Build*** toolbar button options. Also, generate the documentation using the ***Generate Documentation*** option.

This will provide you with the starting point SPI Controller model for this lab.

Task-2: Add a Specification Version Control Parameter

***Motivation**: In this lab, we want to develop a newer version of the SPI Controller but still maintain the older version as well. To be able to do that, this task describes how to associate the model to a specific version of a hardware IP.*

Before we add new features to the model, we will associate the current version of the model to the hardware specification version 1.0.3

3. Go to the ***Parameters*** tab, click on the table and use <*CTRL-n*>, Click the ***Add Parameter*** option from the context menu, or use the ***Add*** icon ( ), or double-click to add a new entry on the table. You can use <*TAB*> to move to the next field in the table.

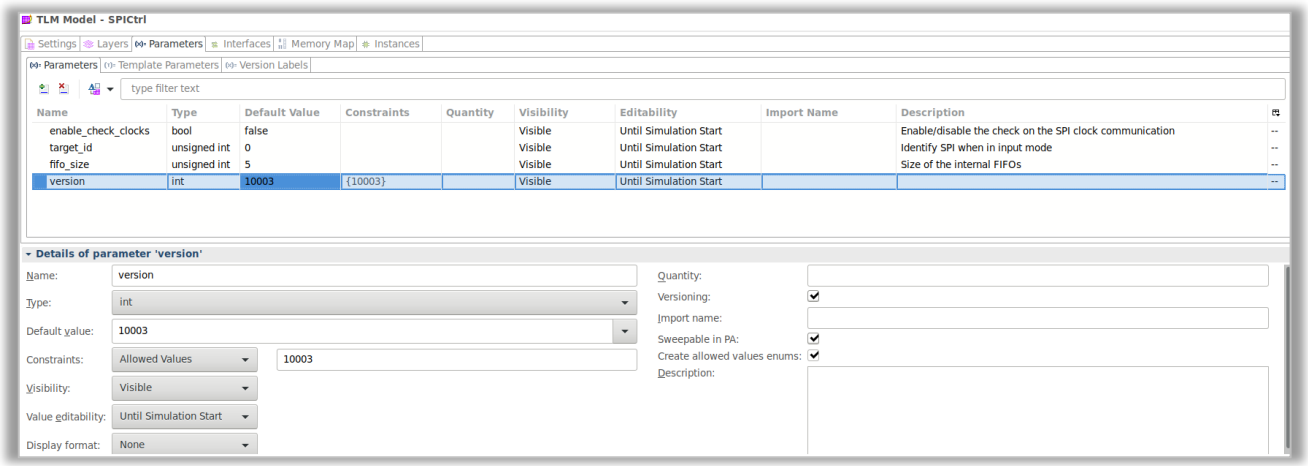
   i. Add a parameter with ***Name*** `version`, ***Type*** `int` and ***Default value*** `10003`.

   ii. Expand the ***Details of parameter 'version'*** panel to see all other options possible. Click to enable the ***Versioning*** checkbox to indicate that this parameter is related to the version selection.

   iii. In the ***Constrains*** listbox, select ***Allowed Values,*** and add the value `10003`.

   Note: Currently TLMC supports only plain integer number as the version number. This may get refined to allow finer control format such as `X.YY.ZZ`.

   iv. Set the ***Default value*** to `10003`.



4. In the ***Memory Map*** tab, click the ***Show Versioned View*** button ( ) on the top.

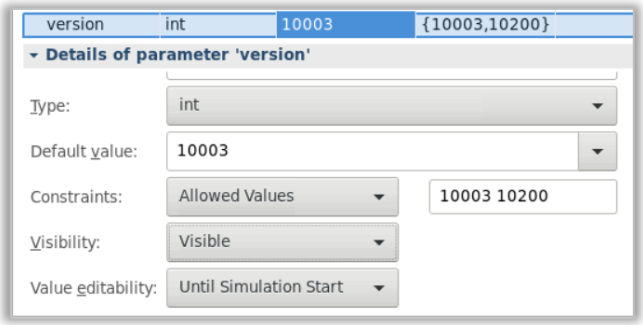
You will see `version_10003` as a possible filter option to show only the memory elements that exist in this particular version.

Task-3: Updating the Model Version

***Motivation**: This task explains how to combine design elements and functionality of a new version of the hardware IP in the same model project.*

Let's imagine that we need to update the model to include CRC functionality available in another version of the hardware IP, version 1.2.0. This functionality allows calculating the CRC of the data received by the SPI Controller. In case a CRC error is found, it is flagged in a new bitfield in the *SR* register.

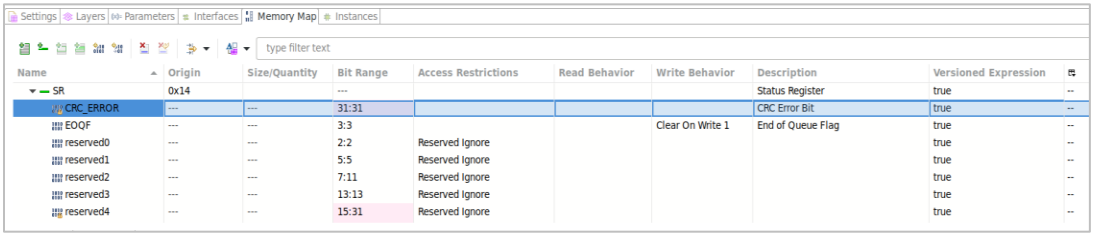
5. The first step is to add the new version tag to the `version` parameter.
   Open the ***Details of parameter 'version'*** section and add the new value in the textbox next to the Allowed values with a space. You can specify multiple space separated values for the version parameter.

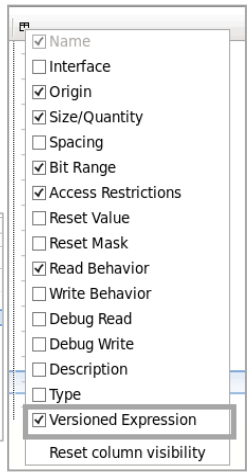


The label automatically gets added to the ***Active Version*** filter in the ***Memory Map*** page as follows.

6. In the ***Memory Map*** page, go to the *SR* register and add a *CRC_ERROR* bitfield with bit range `31:31` and description as "*CRC Error bit*".

However, that bitfield should only exist in the version 1.2.0 of the model. We need to add that information using the ***Versioned Expression*** column.
Note: you may need to add the column by clicking in the ***Customize Table*** icon ( ) on the right-hand side of the table header and selecting the checkbox for ***Versioned Expression***.





Initially all entries in the ***Versioned Expression*** column are marked as `true`. For *CRC_ERROR* bitfield, replace the value `true` (indicating that is valid for all versions), with the expression `version >= 10200` to indicate that is only valid for version 1.2.0 or above. Similarly, the *reserved4* bitfield should only be valid for versions lower than 1.2.0. Finally, add the last reserved bitfield with name *reserved5*, range `15:30` and valid only for version 1.2.0 or above.

Name	Origin	Size/Quantity	Bit Range	Access Restrictions	Read Behavior	Write Behavior	Description	Versioned Expression	
RSER	0x18		---				Request Select and Enable Register	true	--
SR	0x14		---				Status Register	true	--
CRC_ERROR	---	---	31:31				CRC Error Bit	version >= 10200	--
EOQF	---	---	3:3			Clear On Write 1	End of Queue Flag	true	--
reserved0	---	---	2:2	Reserved Ignore				true	--
reserved1	---	---	5:5	Reserved Ignore				true	--
reserved2	---	---	7:11	Reserved Ignore				true	--
reserved3	---	---	13:13	Reserved Ignore				true	--
reserved4	---	---	15:31	Reserved Ignore				version < 10200	--
reserved5	---	---	32:32					version >= 10200	--
RFDF	---	---	14:14			Clear On Write 1	Receive FIFO Drain Flag	true	--
RFOF	---	---	12:12			Clear On Write 1	Receive FIFO Overflow Flag	true	--
TCF	---	---	0:0			Clear On Write 1	Transfer Complete Flag	true	--
TFFF	---	---	6:6			Clear On Write 1	Transmit FIFO Fill Flag	true	--
TFUF	---	---	4:4			Clear On Write 1	Transmit FIFO Underflow Flag	true	--
TXRXS	---	---	1:1			Clear On Write 1	Transmit and Receive Status Flag	true	--

Note: TLM Creator produces few warnings that the *CRC_ERROR*, *reserved4* and *reserved5* bitfields overlap, and highlight the overlaps in red in the table. Those warnings can be safely ignored.

- Warnings (3 items)
  - Model Configuration Problems (3 items)
    - Bitfield 'reserved5', Start Bit: Bitfields overlap
    - Bitfield 'reserved4', Start Bit: Bitfields overlap
    - Bitfield 'CRC_ERROR', Start Bit: Bitfields overlap

The memory map elements can be filtered according to the active version using the ***Show Versioned View*** icon ( ). Hence if the active (i.e. ticked) version is selected as `version_10003` and the ***Showed Versioned View*** icon is set, then the *CRC_ERROR* and reserved5 bitfields will not be shown.

7. The next step is to add an *action* object called *CRC_MOD* to the top-level model in the ***Instances*** tab.

Settings | Layers | Parameters | Interfaces | Memory Map | Instances

type filter text

Name	Modeling Object	Quantity	Description
bus_reg_in_router	router		
TX_PATH	hierarchy		
RX_PATH	hierarchy		
SHIFT_REGISTER	variable		
CRC_MOD	action		

Details of instance 'CRC_MOD'

General | Attributes | Callbacks | Behavior

type filter text

Name	Enabled
main	true

Details of callback 'main'

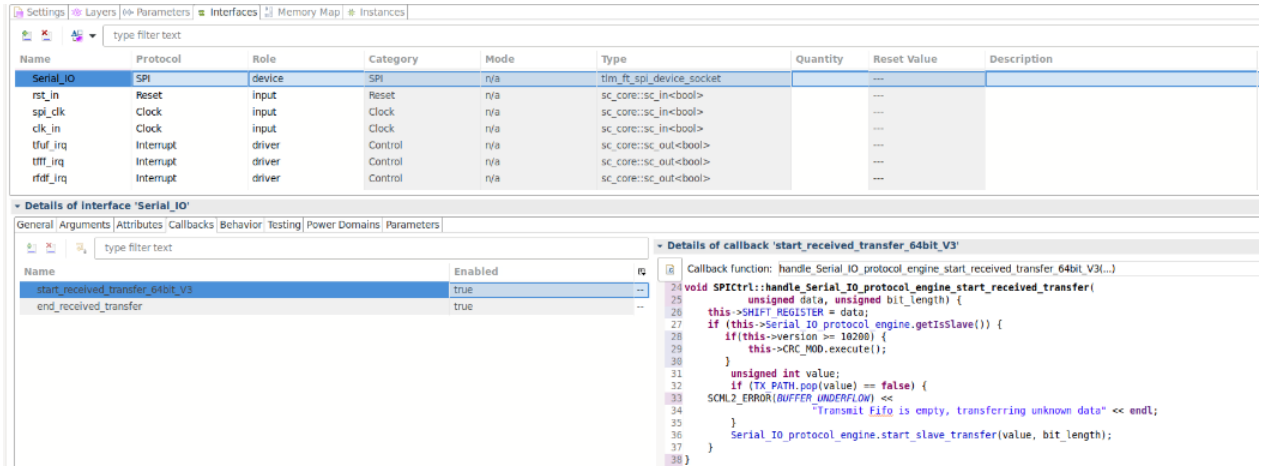
Callback function: handle_CRC_MOD_main(...)

```
void SPICtrl::handle_CRC_MOD_main() {
  this->SR.CRC_ERROR = 0;
  SCML2_MODEL_INTERNAL(LEVEL0)
     << "Performing CRC calculation..." << std::endl;
}
```

Open the ***Callbacks*** tab and modify the main callback as shown below.
Note that a FastTrack message is added, which can be used to check the model functionality by the developer.

```
void SPICtrl::handle_CRC_MOD_main() {
   this->SR.CRC_ERROR = 0;
   SCML2_MODEL_INTERNAL(LEVEL0)
       << "Performing CRC calculation..." << std::endl;
}
```

8. Finally, we will add the CRC functionality to the SPI protocol engine `start_received_transfer_64bit_V3` callback of the `Serial_IO` interface from the ***Interfaces*** tab as shown below.
   Note that the CRC functionality (and the *CRC_ERROR* bitfield) should only exist when the model version is above 1.2.0. Therefore, we guard that code with a version check.



9. Save and Build the project.

What's Next

In the next lab, we will start testing the model using unit tests and coverage.